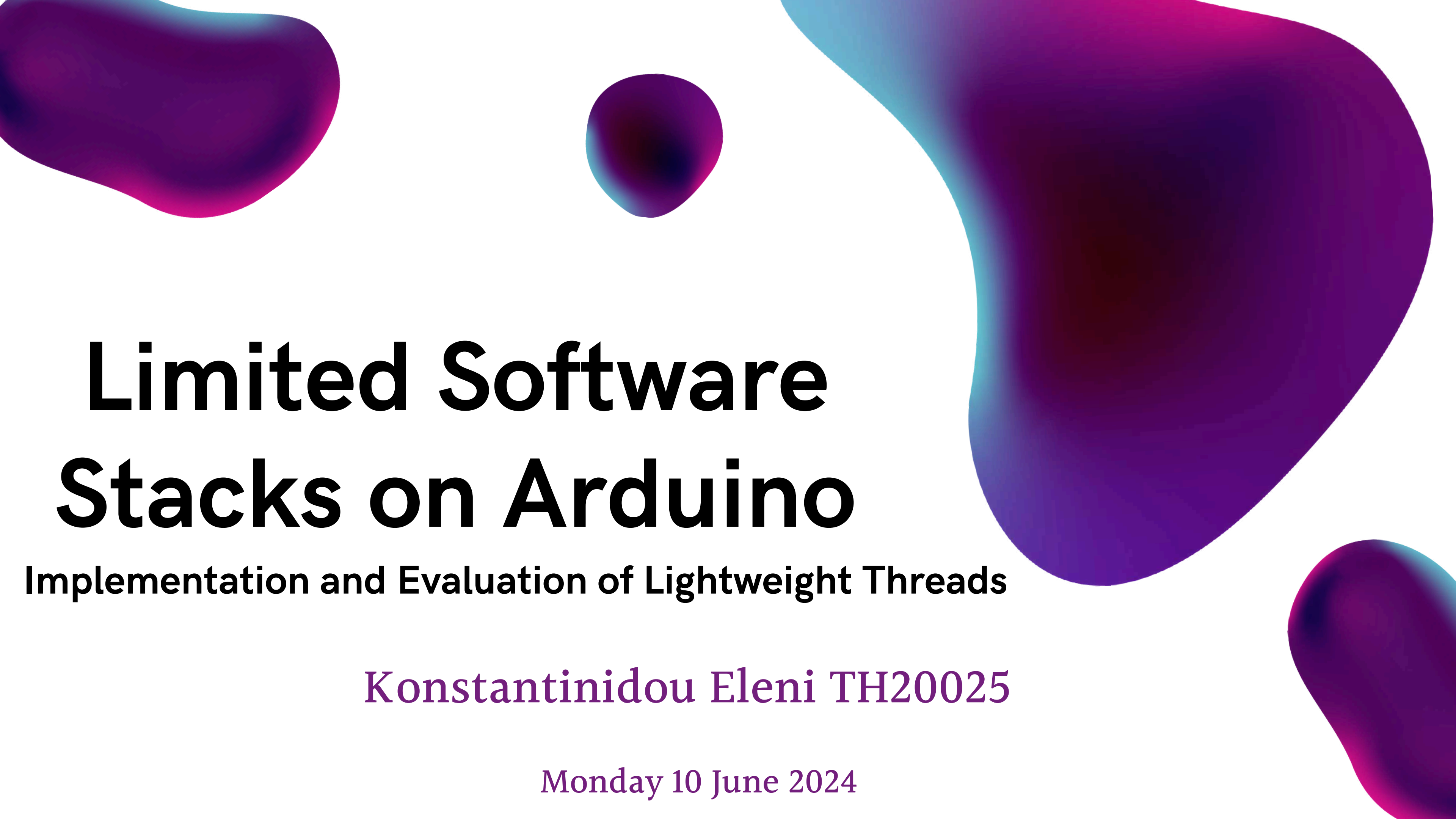


The background features several abstract, organic shapes in shades of purple and blue. A large, irregular shape is on the right side, with a smaller circle above it. There are also some smaller, more defined shapes on the left and bottom right.

Limited Software Stacks on Arduino

Implementation and Evaluation of Lightweight Threads

Konstantinidou Eleni TH20025

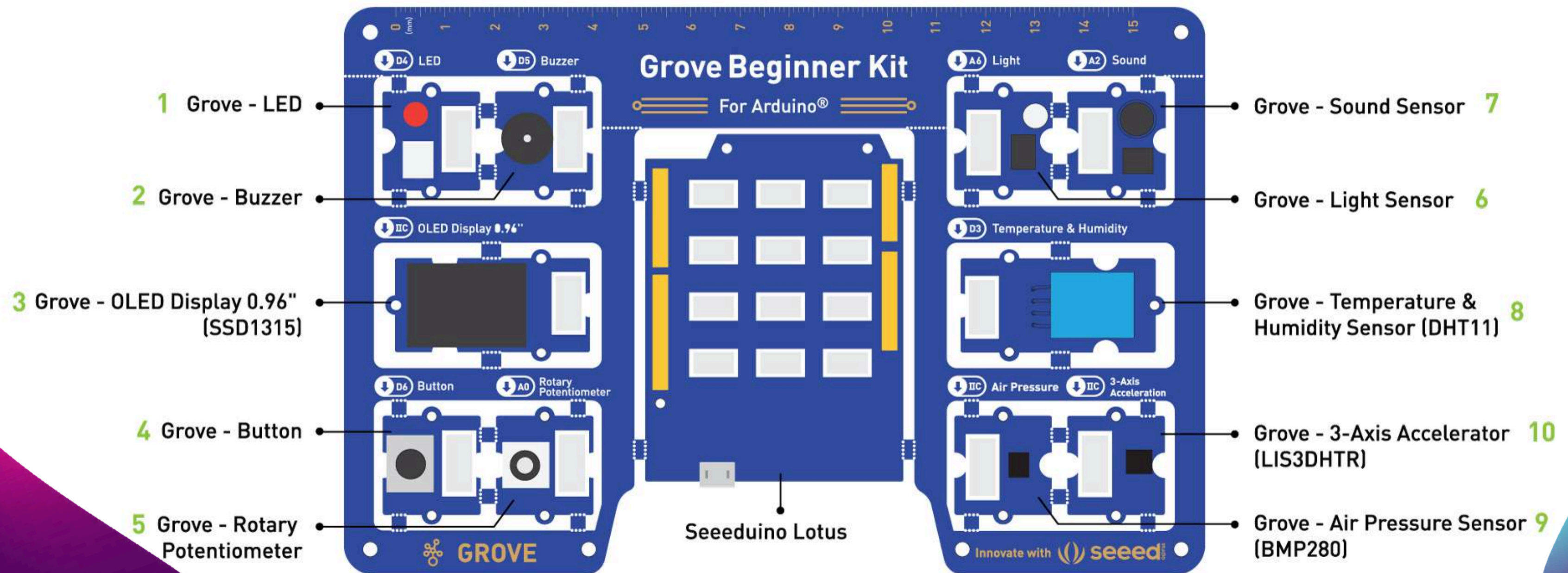
Monday 10 June 2024

Objectives:

- Exploration of lightweight threads (protothreads) on Arduino
- Evaluate performance metrics:
 - communication
 - synchronization
 - context switch latency

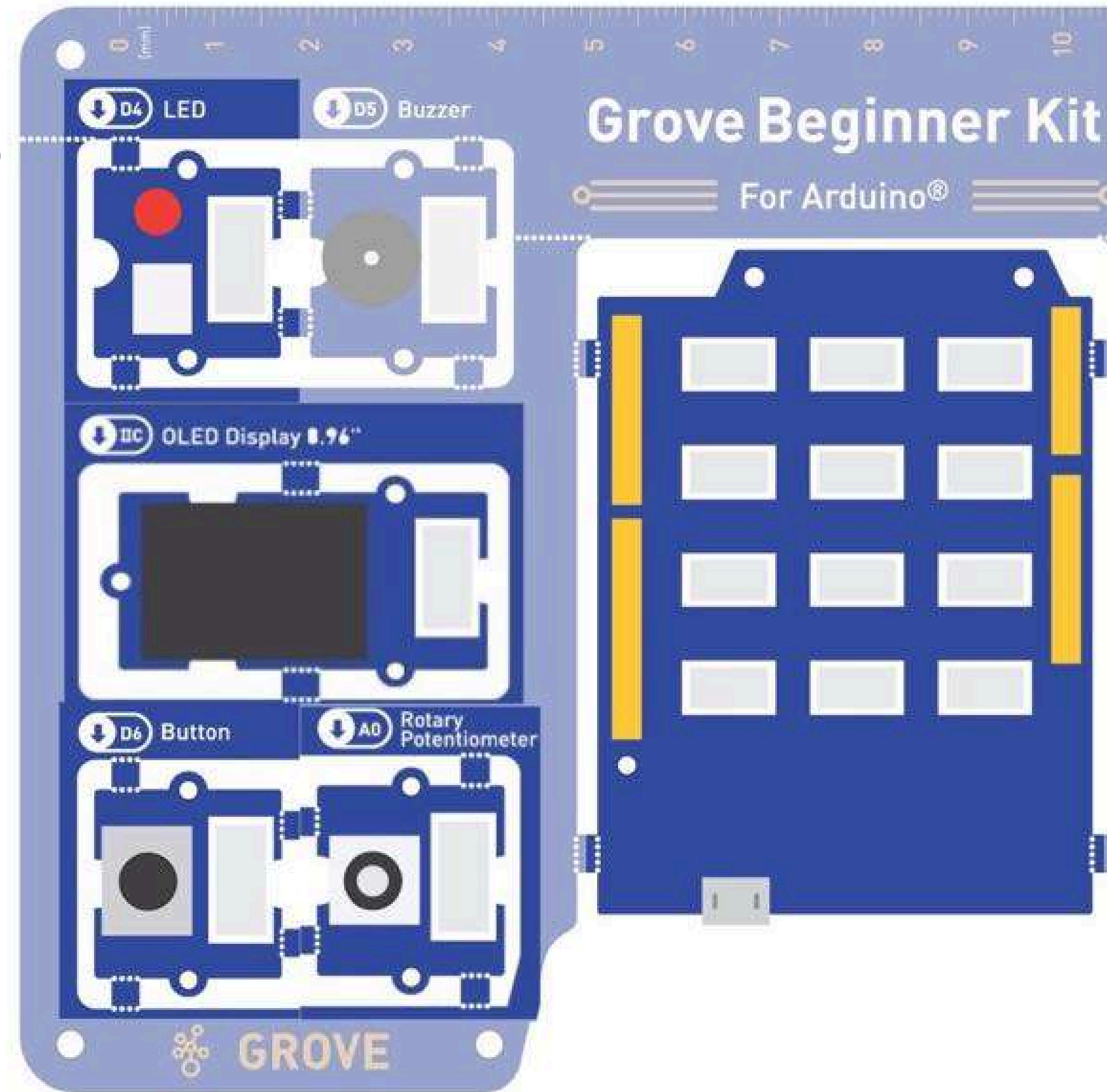
Hardware and Software Setup

The design and the implementation of lightweight threads regarding this project were explored on the Arduino Grove Beginner Kit



Components Used

- **Button:** Toggles state displayed on OLED
- **Potentiometer:** Adjust values that are displayed on the OLED
- **OLED Display:** Shows button state and potentiometer values
- **LED:** Turns ON/OFF based on the potentiometer values



Project Implementation

The main idea of the project implementation is to handle multiple concurrent tasks—specifically, reading button inputs, reading potentiometer values, managing LED states and updating an OLED display. This is achieved with protothreads and semaphores

In more detail the functionality description:

- We change the button's state (ON/OFF)
- The button's state is displayed on the OLED
- We adjust the potentiometer and its values are displayed directly on the OLED
- Depending on the values of the potentiometer, the LED is adjusted (Turned On/ Off)

Local Continuations (LC)

Protothreads are implemented using local continuations.

A local continuation represents the current state of execution at a particular place in the program, but does not provide any call history or local variables. It can be set in a specific function to capture the state of the function. After a local continuation has been set can be resumed in order to restore the state of the function at the point where the local continuation was set.

Local continuations can be implemented in a variety of ways:

- by using machine specific assembler code
- by using standard C constructs
- by using compiler extensions.

Lightweight Threads: Protothreads

Protothreads are a lightweight, stackless form of threading designed for use in memory-constrained environments, such as embedded systems and microcontrollers. They provide a way to manage multiple tasks concurrently without the overhead and complexity of traditional multi-threading.

Key Characteristics and Benefits:

- Cooperative Multitasking
- Lightweight and Low Memory Overhead
- Simplicity in Synchronization
- Ease of Use

Protothread Implementation

- ▶ First step is to initialize the necessary protothreads for each task that we are going to implement
- ▶ We continue by writing each task's protothread function

```
void setup(){  
  //LED_PIN acts as an output meaning is used to send signals  
  pinMode(LED_PIN, OUTPUT);  
  
  //BUTTON_PIN acts as in input meaning is used to receive signals  
  pinMode(BUTTON_PIN, INPUT);  
  
  Serial.begin(38400); //baud  
  u8x8.begin();  
  u8x8.setFont(u8x8_font_chroma48medium8_r); //defining the  
  font  
  
  //initializing the protothreads that we are using  
  PT_INIT(&pt_button);  
  PT_INIT(&pt_potentiometer);  
  PT_INIT(&pt_led);  
  PT_INIT(&pt_display);  
}
```

Protothread Button Function

//protothread function about the button. Handles button inputs

```
static int protothread_button(struct pt *pt) {
```

```
    PT_BEGIN(pt); //beginning of the protothread
```

```
    static int buttonState = 0; //declaring the button state
```

```
    while (1) {
```

```
        startTime = micros(); //starting recording the time
```

```
        int currentState = digitalRead(BUTTON_PIN);
```

```
        if (currentState != buttonState) {
```

```
            lastDebounceTime = millis();
```

```
        }
```

```
        if ((millis() - lastDebounceTime) > debounceDelay) {
```

```
            if (currentState == HIGH) {
```

```
                ledState = !ledState;
```

```
            }
```

```
    }
```

```
        buttonState = currentState;
```

```
        endTime = micros();
```

```
        Serial.print("Button Protothread Time: ");
```

```
        Serial.println(endTime - startTime);
```

```
        PT_YIELD(pt); //allows other  
        protothreads to run
```

```
    }
```

```
    PT_END(pt); //end of the protothread
```

```
}
```

Protothread Potentiometer Function

```
//protothread function about the potentiometer. Handles potentiometer readings  
//Reads the value from the potentiometer
```

```
static int protothread_potentiometer(struct pt *pt) {  
    PT_BEGIN(pt); //beginning of the protothread  
  
    while (1) {  
        startTime = micros(); //starting recording the time  
        potValue = analogRead(POTENTIOMETER_PIN); //reading potentiometer value  
        endTime = micros(); //ending recording the time  
  
        Serial.print("Potentiometer Protothread Time: ");  
        Serial.println(endTime - startTime); //printing the calculating potentiometer protothread  
        execution time  
        PT_YIELD(pt); //allows other protothreads to run  
    }  
    PT_END(pt); //end of the protothread  
}
```

Protothread LED Function

//protothread function about the led. Manages LED state changes based on the potentiometer value and the button state.

```
static int protothread_led(struct pt *pt) {  
    PT_BEGIN(pt); //beginning of the protothread  
  
    while (1) {  
        startTime = micros(); //starting recording the time  
        analogWrite(LED_PIN, ledState ? map(potValue, 0, 1023, 0, 255) : 0);  
        endTime = micros(); //ending recording the time  
  
        Serial.print("LED Protothread Time: ");  
        Serial.println(endTime - startTime); //printing the execution time regarding the led's  
        protothread time  
  
        PT_YIELD(pt); //allows other protothreads to run  
    }  
    PT_END(pt); //end of the protothread }
```



Protothread OLED Function



//protothread function about the oled display.
Manages updates to the display with the LED state
and potentiometer value

```
static int protothread_display(struct pt *pt) {  
    PT_BEGIN(pt); //beginning of the protothread
```

```
    while (1) {  
        startTime = micros();  
        if (ledState != lastLedState || potValue !=  
lastPotValue) {  
            // Only update the parts of the display that  
have changed
```

```
        if (ledState != lastLedState) {  
            u8x8.setCursor(0, 0);  
            u8x8.print("LED State: ");  
            u8x8.print(ledState ? "ON " : "OFF");  
            lastLedState = ledState; }  
    }
```

```
        if (potValue != lastPotValue) {  
            u8x8.setCursor(0, 1);  
            u8x8.print("Pot Value: ");  
            u8x8.print(potValue);  
            lastPotValue = potValue;  
        }
```

```
    }  
    endTime = micros(); //ending recording time  
    Serial.print("Display Protothread Time: ");  
    Serial.println(endTime - startTime);  
    //printing the execution time regarding the oled's  
protothread time
```

```
    PT_YIELD(pt); //allows other protothreads to run  
}
```

```
    PT_END(pt); //end of the protothread  
}
```

loop Function

- The loop() function schedules each protothread, records the cycle start time and calculates the cycle duration.

```
void loop() {  
    cycleStartTime = micros(); //Recording the start time of the cycle  
  
    //macro that manages the execution of a protothread  
    PT_SCHEDULE(protothread_button(&pt_button));  
    PT_SCHEDULE(protothread_potentiometer(&pt_potentiometer));  
    PT_SCHEDULE(protothread_led(&pt_led));  
    PT_SCHEDULE(protothread_display(&pt_display));  
  
    //Calculating and printing the elapsed time for the cycle  
    unsigned long cycleEndTime = micros(); //ending the cycle's time recording  
    Serial.print("Full Cycle Time: ");  
    Serial.println(cycleEndTime - cycleStartTime); //calculating and printing the full  
    cycle's elapsed time  
}
```

Semaphores

Semaphores are synchronization tools used in concurrent programming to control access to shared resources by multiple processes or threads. The main aim of using a semaphore is process synchronization and access control for a common resource in a concurrent environment.

Key Characteristics and Benefits:

- Avoidance of Race Conditions
- Synchronization of Tasks
- Resource Management
- Deadlock Prevention
- Flexibility in Implementation

Semaphores Implementation

- Initialize the necessary semaphores for each task

```
void setup(){  
  //LED_PIN acts as an output meaning is used to send signals  
  pinMode(LED_PIN, OUTPUT);  
  
  //BUTTON_PIN acts as in input meaning is used to receive  
signals  
  pinMode(BUTTON_PIN, INPUT);  
  Serial.begin(38400); //baud  
  
  u8x8.begin();  
  u8x8.setFont(u8x8_font_chroma48medium8_r);  
  
  //initializing the protothreads that we are using  
  PT_INIT(&pt_button);  
  PT_INIT(&pt_potentiometer);  
  PT_INIT(&pt_led);  
  PT_INIT(&pt_display);  
  
  //initializing the semaphores  
  PT_SEM_INIT(&sem_button, 1);  
  PT_SEM_INIT(&sem_potentiometer, 1);  
  PT_SEM_INIT(&sem_led, 1);  
  PT_SEM_INIT(&sem_display, 1);  
}
```

- We continue by writing each task's semaphore function

Semaphore Button Function

//protothread function about the button. Handles button inputs

```
static int protothread_button(struct pt *pt) {  
    PT_BEGIN(pt); //beginning of the protothread  
    static int buttonState = 0; //declaring the button state  
  
    while (1) {  
        PT_SEM_WAIT(pt, &sem_button); //wait for the semaphore (enter critical  
section)  
  
        unsigned long interProcessTime = micros() - lastEndTime; // Calculate time  
since last protothread ended  
        Serial.print("Time Between Protothreads (Button): ");  
        Serial.println(interProcessTime);  
  
        startTime = micros(); //starting recording the time  
  
        int currentState = digitalRead(BUTTON_PIN); //reading the button state
```

```
if (currentState != buttonState) {
    lastDebounceTime = millis(); //resetting debounce timer if the state changes
}
//checking the debounce delay
if ((millis() - lastDebounceTime) > debounceDelay) {
    if (currentState == HIGH) {
        ledState = !ledState; //toggle led state
    }
}
buttonState = currentState; //updating the button state

endTime = micros();
Serial.print("Button Protothread Time: ");
Serial.println(endTime - startTime);
//calculating and printing the execution time of the button prot. function

PT_SEM_SIGNAL(pt, &sem_button); //signaling the semaphore (exit critical section)

lastEndTime = endTime; // Update the last end time to current end time
PT_YIELD(pt); //allows other protothreads to run
}
PT_END(pt); //end of the protothread
}
```

Semaphore Potentiometer Function

//protothread function about the potentiometer. Handles potentiometer readings

```
static int protothread_potentiometer(struct pt *pt) {
```

```
    PT_BEGIN(pt); //beginning of the protothread
```

```
    while (1) {
```

```
        PT_SEM_WAIT(pt, &sem_potentiometer); //wait for the semaphore (enter critical section)
```

```
        unsigned long interProcessTime = micros() - lastEndTime; // Calculate time since last protothread ended
```

```
        Serial.print("Time Between Protothreads (Potentiometer): ");
```

```
        Serial.println(interProcessTime);
```

```
        startTime = micros(); //starting recording the time
```

```
        potValue = analogRead(POTENTIOMETER_PIN); //analog read the potentiometer's value
```

```
        endTime = micros(); //ending recording the time
```

```
Serial.print("Potentiometer Protothread Time: ");  
    Serial.println(endTime - startTime); //printing the calculating potentiometer  
    protothread execution time
```

```
    PT_SEM_SIGNAL(pt, &sem_potentiometer); //signal the semaphore (exit  
    critical section)
```

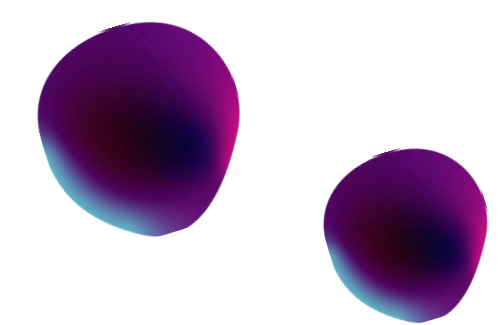
```
        lastEndTime = endTime; // Update the last end time to current end time  
        PT_YIELD(pt); //allows other protothreads to run  
    }
```

```
    PT_END(pt); //end of the protothread  
}
```

Semaphore LED Function

//protothread function about the led. Manages LED state changes

```
static int protothread_led(struct pt *pt) {  
    PT_BEGIN(pt); //beginning of the protothread  
  
    while (1) {  
        PT_SEM_WAIT(pt, &sem_led); //wait for the semaphore (enter critical section)  
  
        unsigned long interProcessTime = micros() - lastEndTime; // Calculate time  
        since last protothread ended  
        Serial.print("Time Between Protothreads (LED): ");  
        Serial.println(interProcessTime);  
  
        startTime = micros(); //starting recording the time  
        analogWrite(LED_PIN, ledState ? map(potValue, 0, 1023, 0, 255) : 0);  
        //updating the LED  
        endTime = micros(); //ending recording the time
```



```
Serial.print("LED Protothread Time: ");  
    Serial.println(endTime - startTime); //printing the execution time regarding  
the led's protothread time
```

```
    PT_SEM_SIGNAL(pt, &sem_led); //signal the semaphore (exit critical  
section)
```

```
        lastEndTime = endTime; // Update the last end time to current end time  
        PT_YIELD(pt); //allows other protothreads to run  
    }
```

```
    PT_END(pt); //end of the protothread  
}
```



Semaphore OLED Function

```
//protothread function about the oled display. Manages updates to the display
static int protothread_display(struct pt *pt) {
    PT_BEGIN(pt); //beginning of the protothread

    while (1) {
        PT_SEM_WAIT(pt, &sem_display); //wait for the semaphore (enter critical section)

        unsigned long interProcessTime = micros() - lastEndTime; // Calculate time since
last protothread ended
        Serial.print("Time Between Protothreads (Display): ");
        Serial.println(interProcessTime);
        startTime = micros(); //starting recording the time

        if (ledState != lastLedState || potValue != lastPotValue) {

            //Only update the parts of the display that have changed
            if (ledState != lastLedState) {
                u8x8.setCursor(0, 0); //setting the cursor's position
```

```
    u8x8.print("LED State: "); //printing the label regarding the LED State
    u8x8.print(ledState ? "ON " : "OFF"); //printing the current LED state
    lastLedState = ledState; //update last LED state
}
if (potValue != lastPotValue) {
    u8x8.setCursor(0, 1);
    u8x8.print("Pot Value: ");
    u8x8.print(potValue);
    lastPotValue = potValue;
}
}
endTime = micros(); //ending recording the time
Serial.print("Display Protothread Time: ");
Serial.println(endTime - startTime); //printing the execution time regarding the
oled's protothread time

    PT_SEM_SIGNAL(pt, &sem_display); //signal the semaphore (exit critical section)

    lastEndTime = endTime; // Update the last end time to current end time
    PT_YIELD(pt); //allows other protothreads to run
}
PT_END(pt); //end of the protothread
}
```

Timing Measurements Protothreads

time taken for each protothread to execute in microseconds (μs)
& the full cycle time for all protothreads

- Button Protothread Time: Takes around 12 μs per cycle. This is a minimal overhead for checking the button state and handling debouncing.
- Potentiometer Protothread Time: This fluctuates slightly between 108 and 116 μs per cycle. This time is spent reading the analog value from the potentiometer.
- LED Protothread Time: Initially, this was around 12 μs , but after some cycles, it increased to 52-64 μs . The change is due to the analogWrite function, which sets the analog output (PWM) value for the LED.
- Display Protothread Time: Initially, it takes around 4 μs , but spikes to over 130,000 μs (130 ms) in some cycles. The spikes occur when the display is updated, which involves communication over Inter-Integrated Circuit and takes significantly longer.
- Full Circle Time: The total time taken to run all protothreads once, is 31,460 μs . However, there are significant spikes up to 161,932 μs during display updates. These spikes indicate that updating the display is the most time-consuming operation and causes delays in the overall cycle.

Timing Measurements Semaphores

time taken for each protothread to execute in microseconds (μs)
& the full cycle time for all protothreads

- Button Protothread Time: Takes around 8-12 μs per cycle. Occasional values slightly higher but within the same range.
- Potentiometer Protothread Time: This consistently takes around 112 μs per cycle.
- LED Protothread Time: Generally around 12-56 μs , with some occasional higher values up to 130,684 μs . The increase in time is due to the analogWrite function, which sets the PWM value for the LED.
- Display Protothread Time: Initially, it takes around 4 μs , but spikes to over 130,000 μs (130 ms) in some cycles. The spikes occur when the display is updated, which involves communication over Inter-Integrated Circuit (I2C) and takes significantly longer.

Timing Measurements Semaphores (2)

Full Cycle Time: The total time taken to run all protothreads once is 76,440 μ s. However, there are significant spikes up to 200,036 μ s during certain display updates. These spikes indicate that updating the display is the most time-consuming operation and causes delays in the overall cycle.

Time Between Protothreads (Context Switch Time): The times between different protothreads are relatively consistent:

- **Button to Potentiometer:** Approximately 13,780 microseconds
- **Potentiometer to LED:** Approximately 7,276 microseconds
- **LED to Display:** Approximately 9,512 microseconds
- **Display to Button:** Approximately 6,712 microseconds

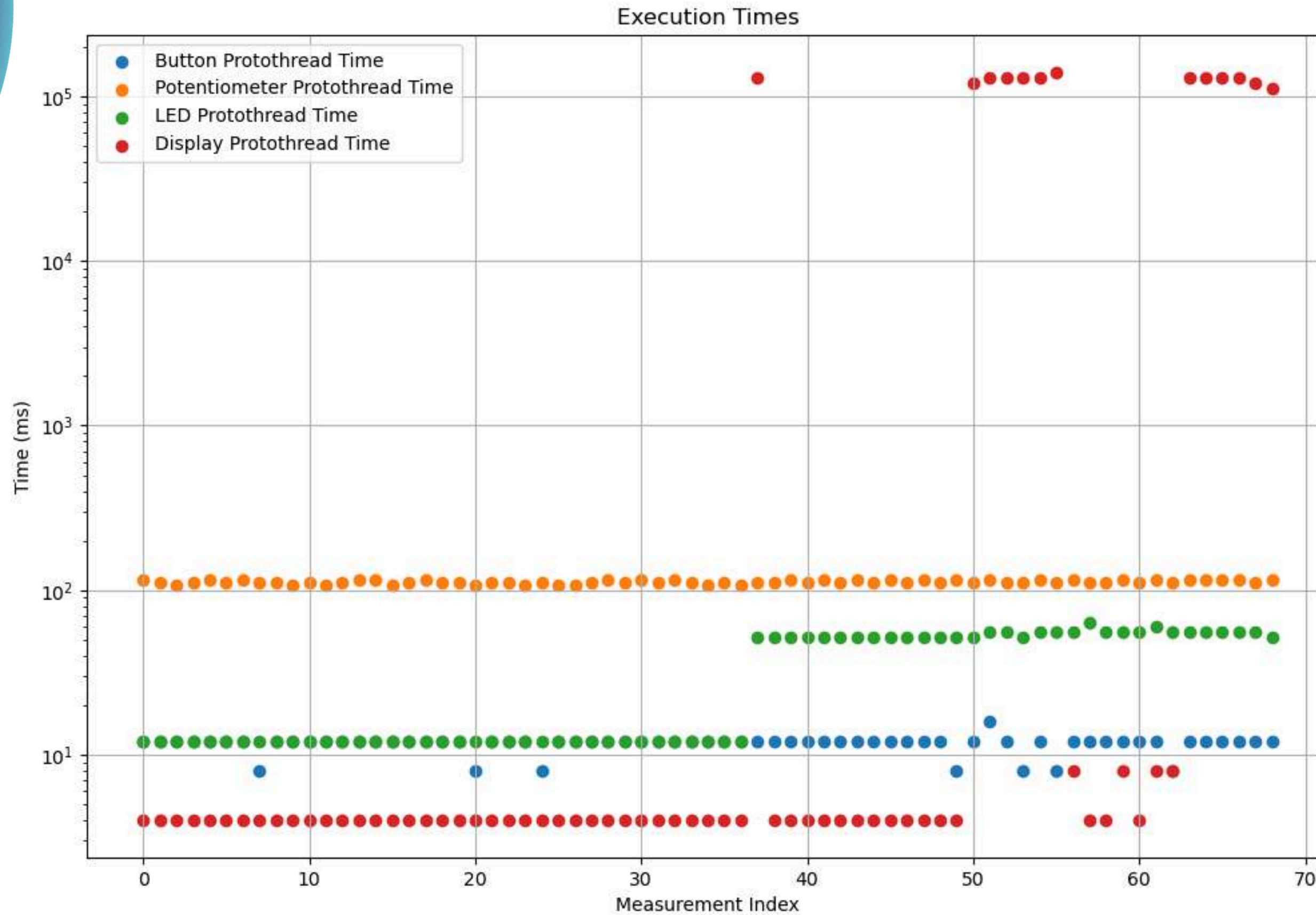
Gathered Timing Measurements

Array with gathered data regarding the execution times for each protothread

	Button Protothread Time (per cycle)	Potentiometer Protothread Time (per cycle)	LED Protothread Time (per cycle)	Display Protothread Time (per cycle)	Full Circle Time (per cycle)
Protothreads code	~ 12 μ s	108 – 116 μ s	Initially 12 μ s Then 52-64 μ s	Initially 4 μ s When updating: 135 μ s	Generally Stable ~31,460 μ s
Code with Semaphores	8 – 12 μ s	112 μ s	12 – 56 μ s	Initially 4 μ s When updating: 130 μ s	Ranging between 73,712 to 76,440

Execution Times Diagram

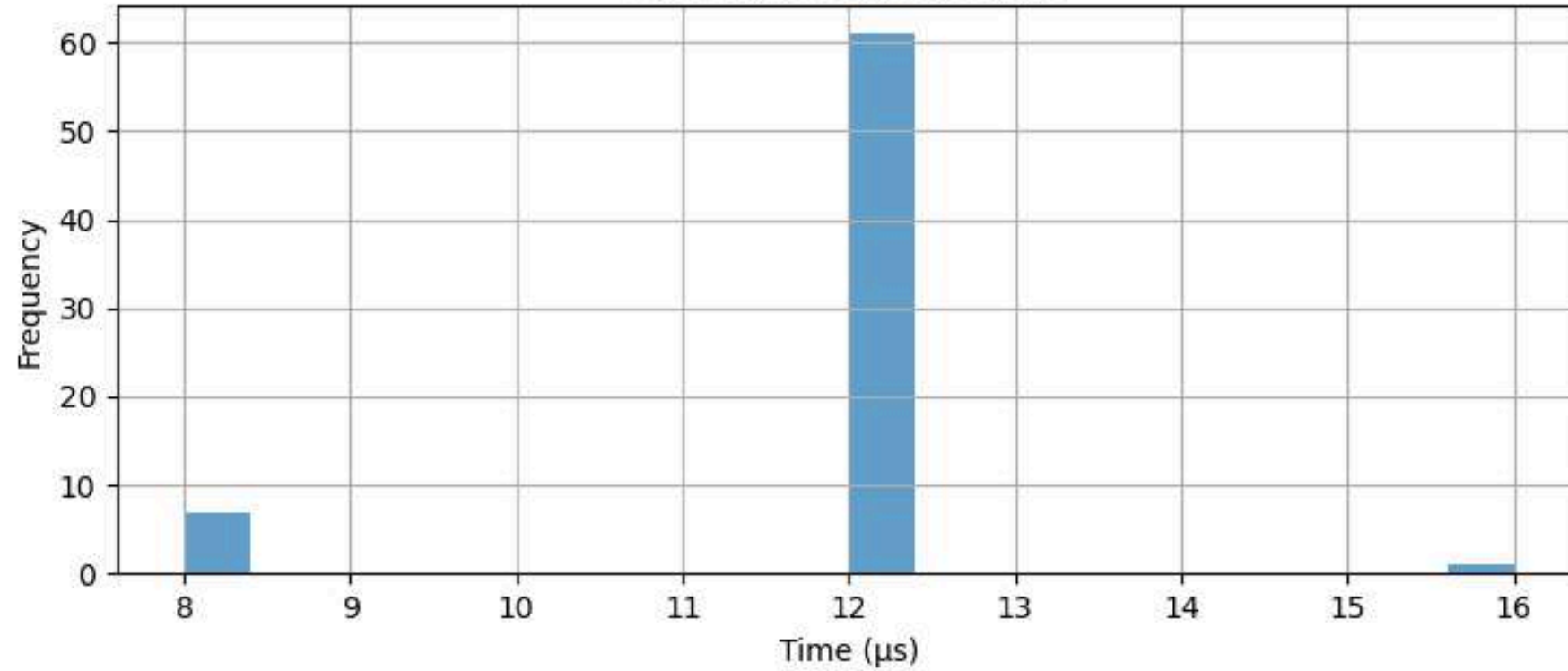
regarding the protothread's timing measurements



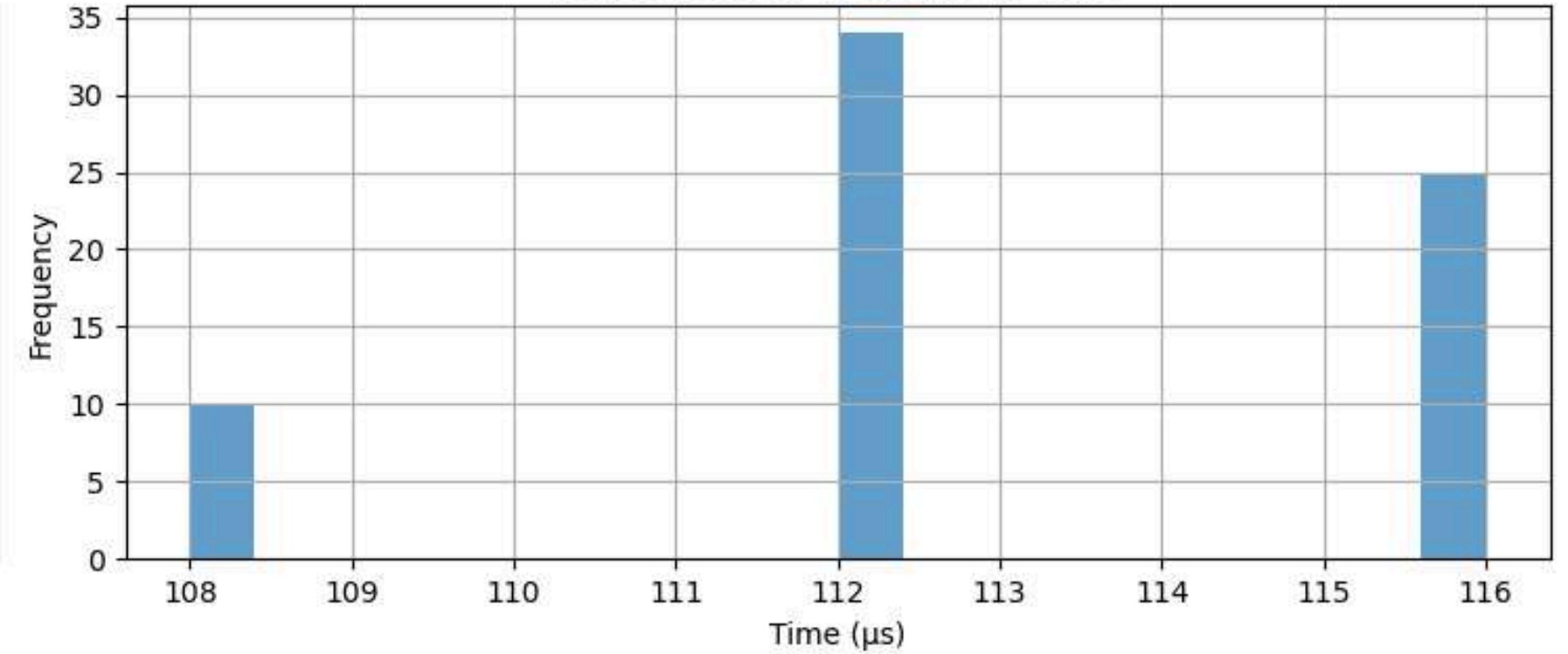
Distribution Histograms

regarding the protothread's timing measurements

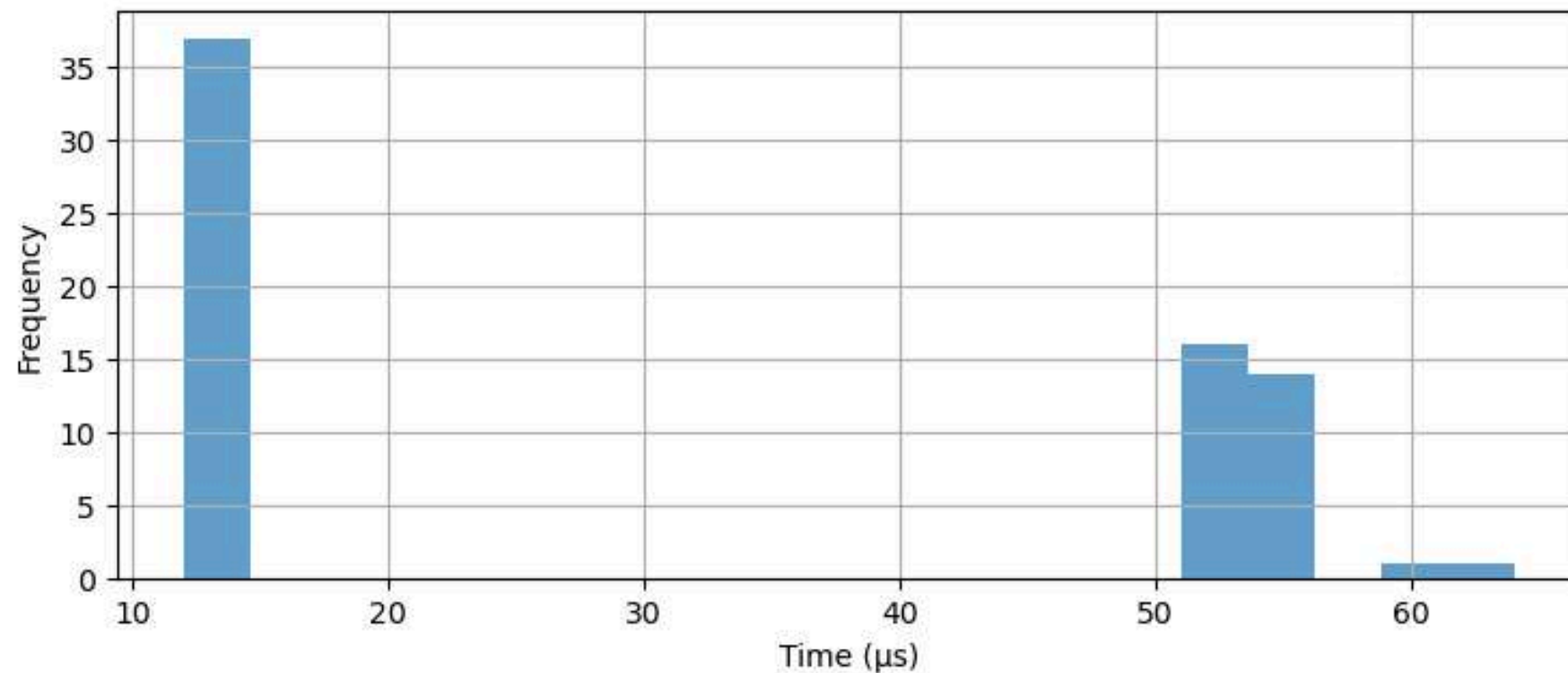
Button Protothread Time



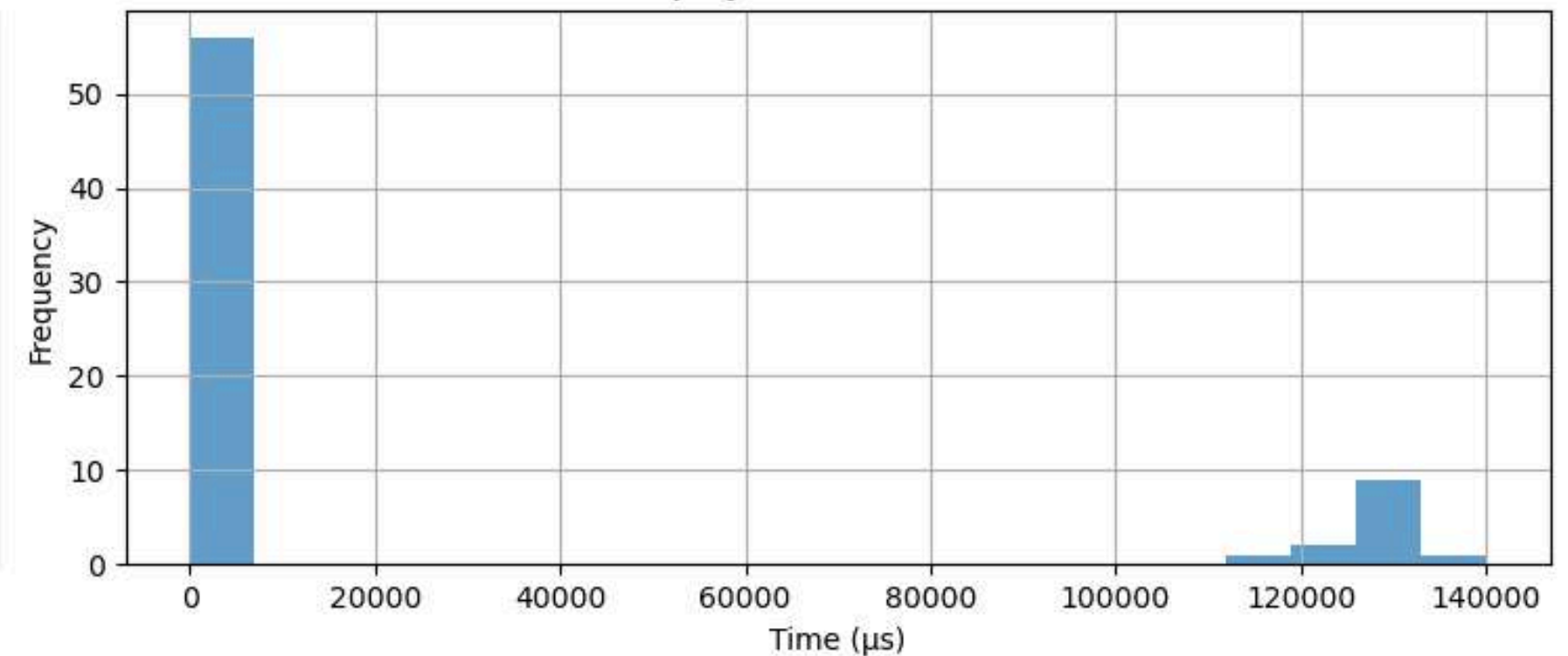
Potentiometer Protothread Time



LED Protothread Time

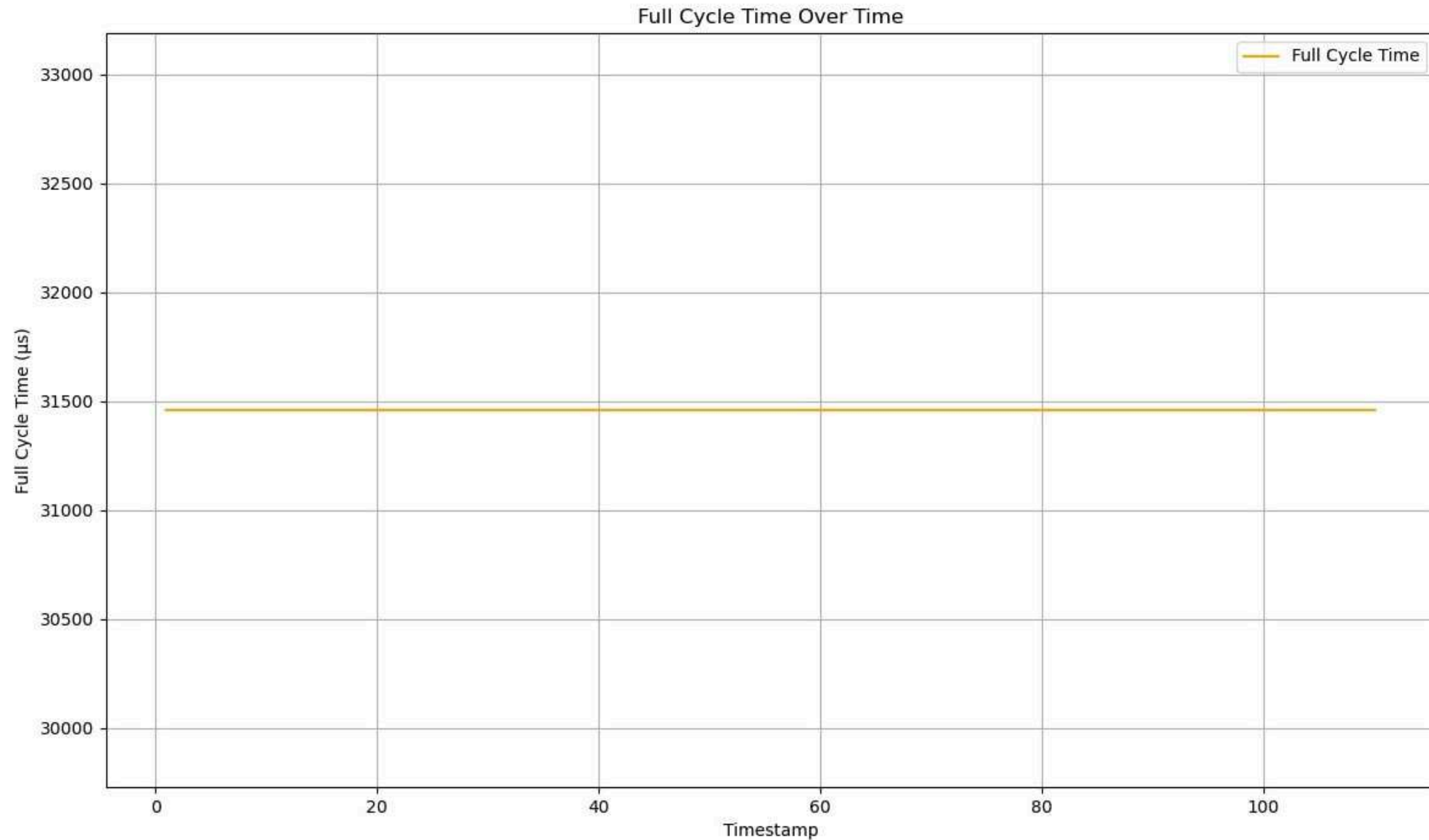


Display Protothread Time



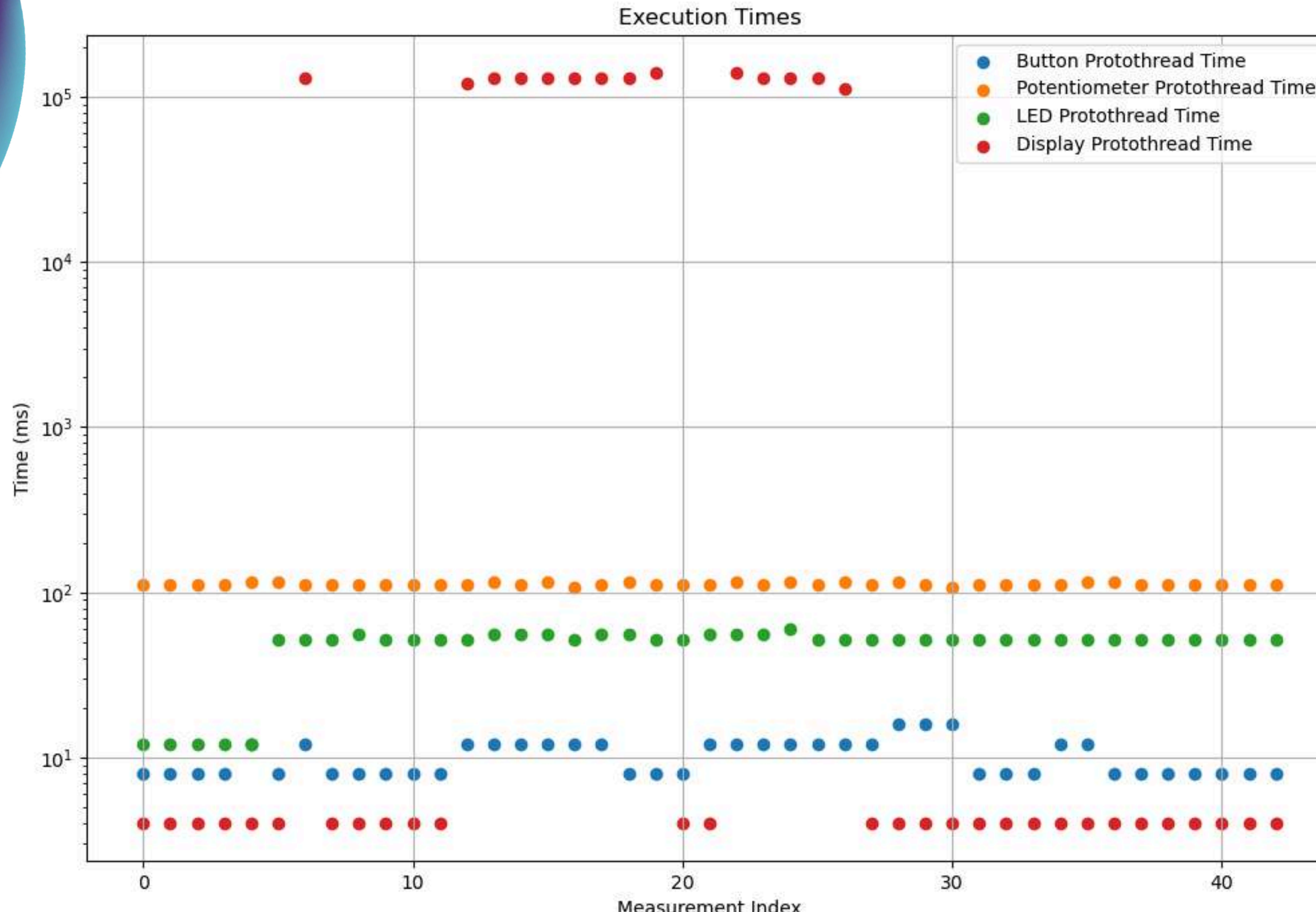
Full Circle Diagram

regarding the protothreads's timing measurements



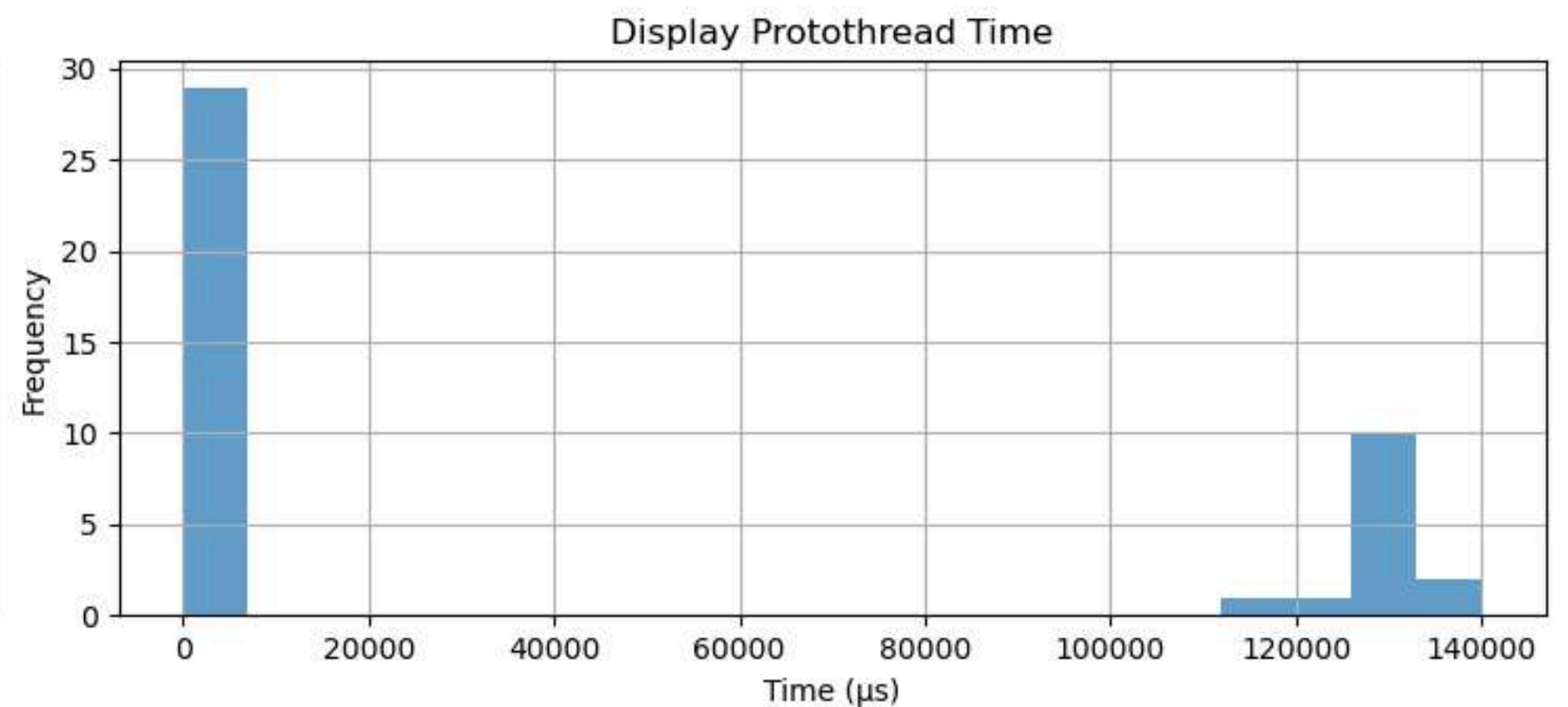
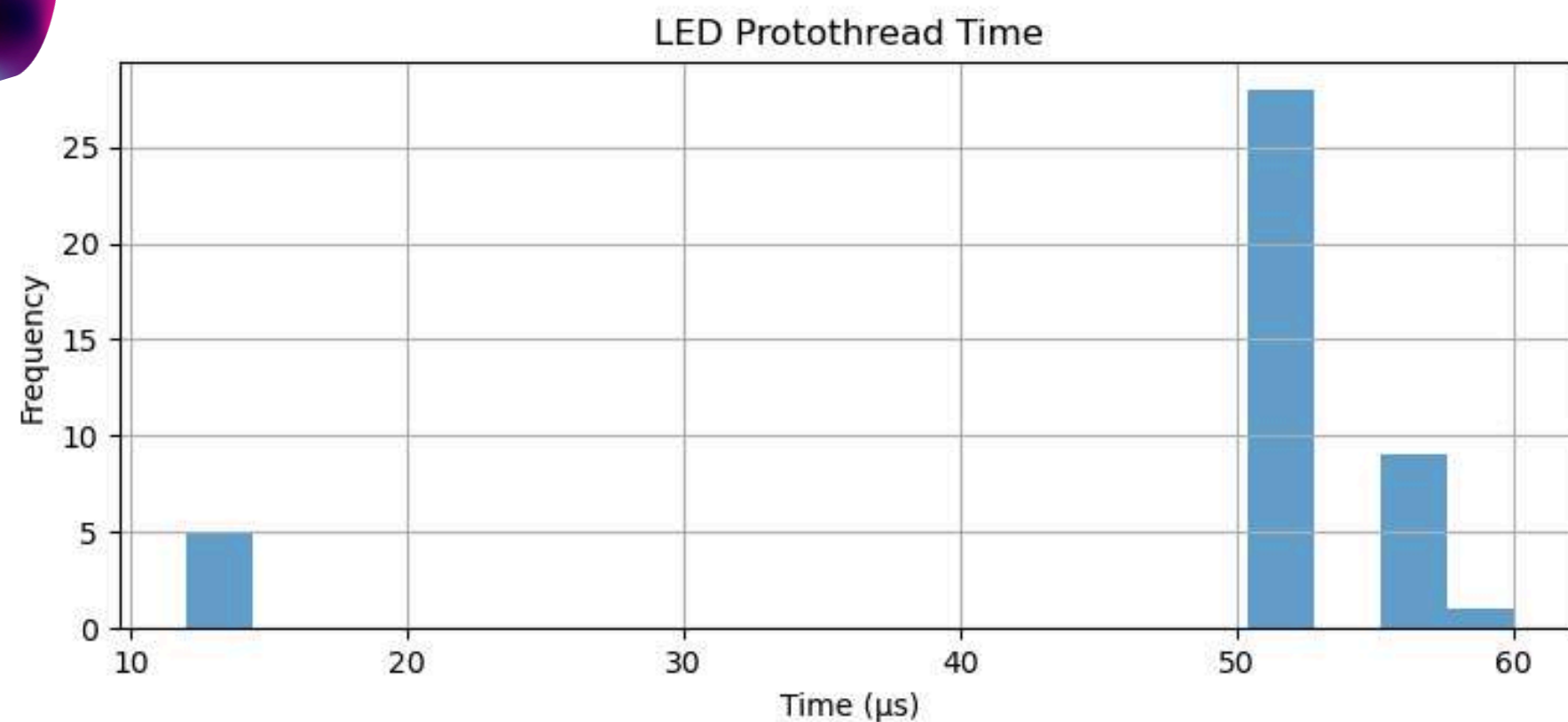
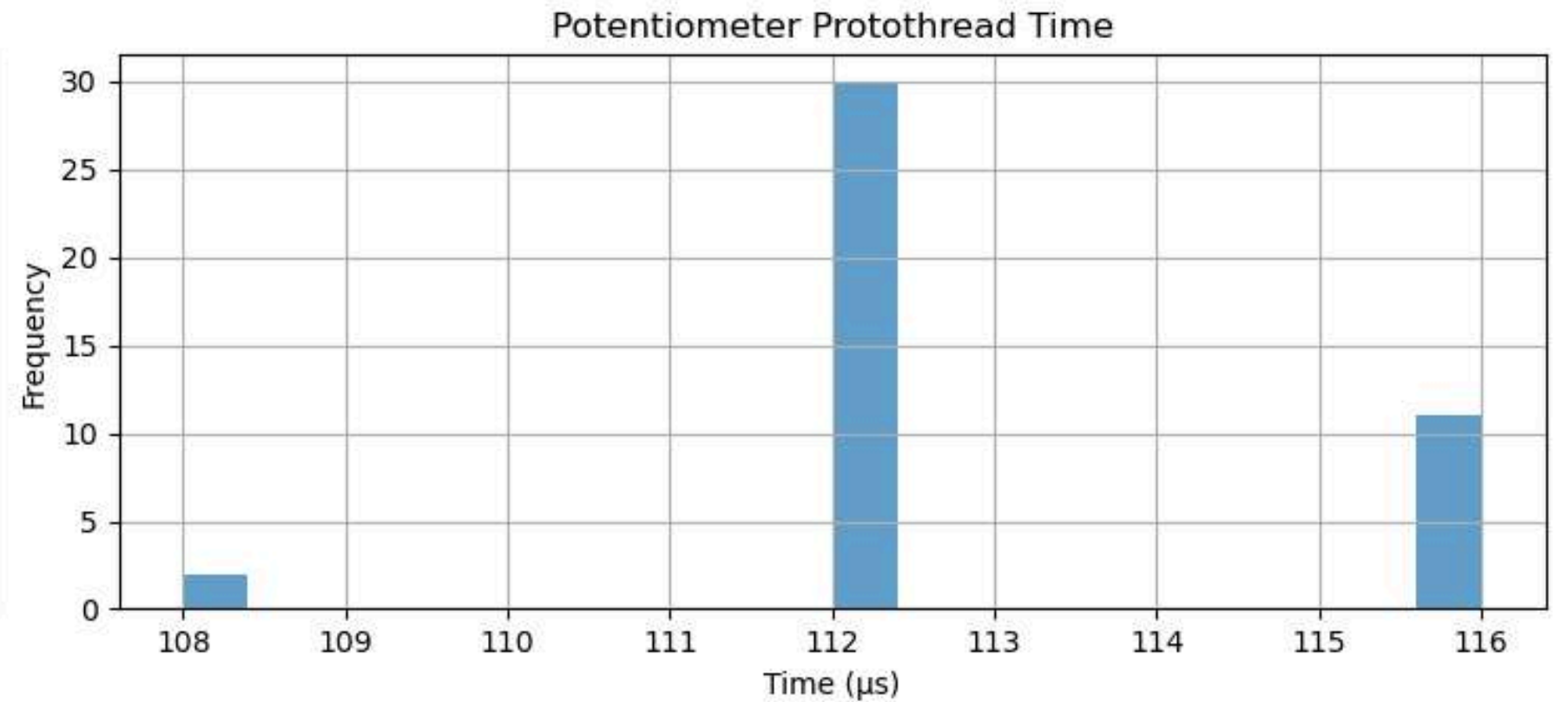
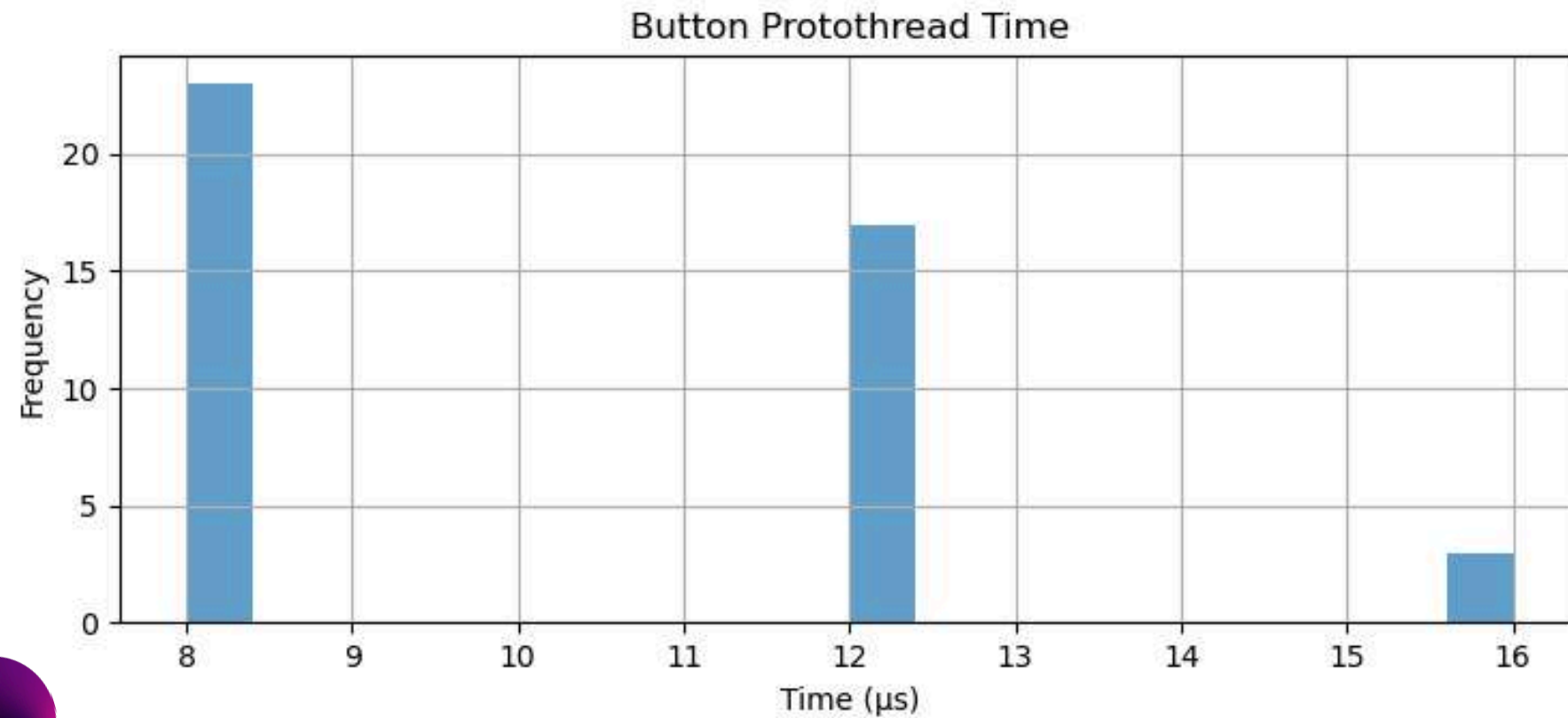
Execution Times Diagram

regarding the semaphore's timing measurements



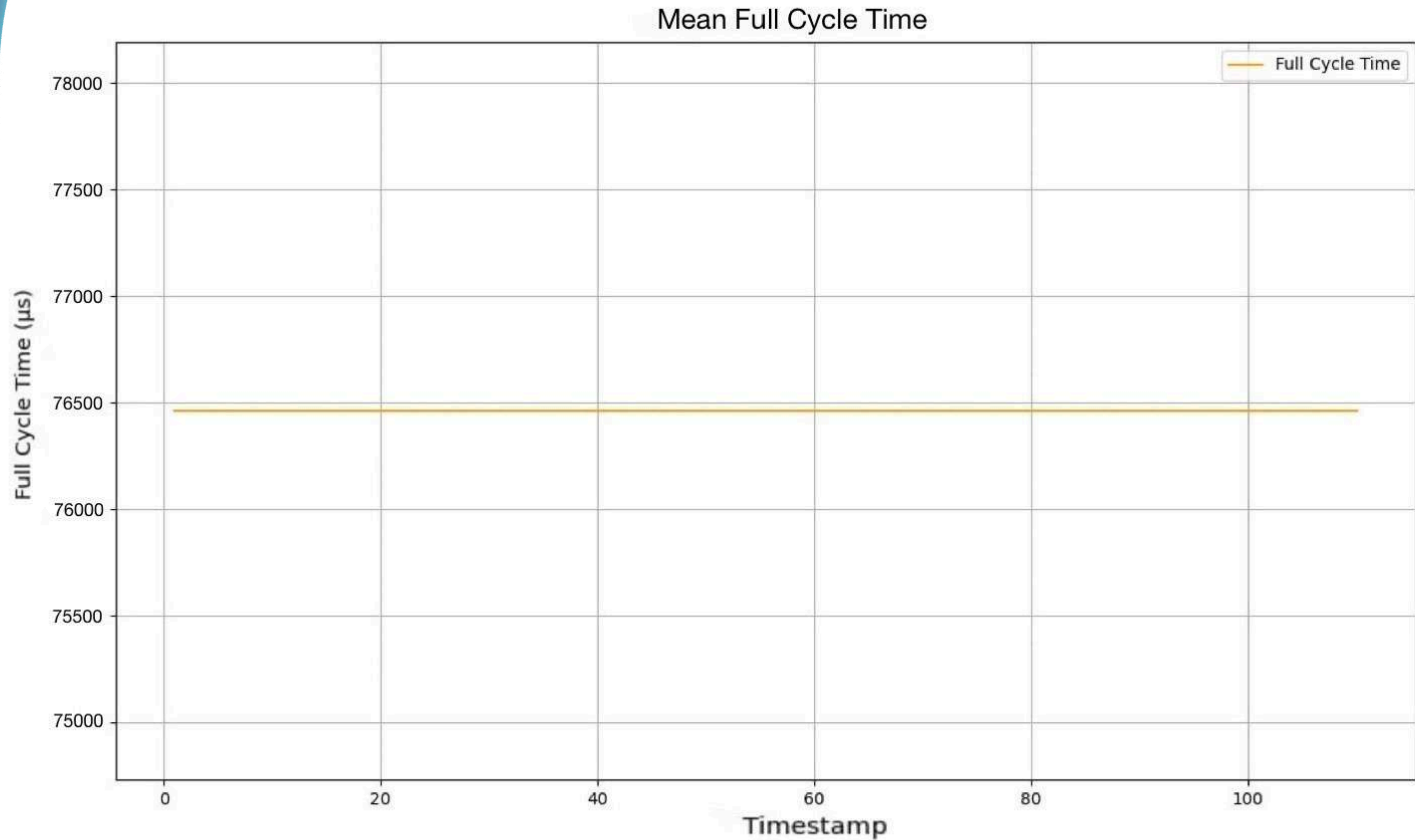
Distribution Histograms

regarding the semaphore's timing measurements



Full Circle Diagram

regarding the semaphore's timing measurements



Comparison of the previous implementations

Button Protothread Time:

- Efficiency: Minimal difference, with a slight improvement in lower range using semaphores.
- Stability: Both methods show consistent performance.

Potentiometer Protothread Time:

- Efficiency: Similar performance, minor increase in time with semaphores.
- Stability: Both methods provide consistent results.

LED Protothread Time:

- Efficiency: Comparable initial times; semaphore implementation shows a slight reduction in the upper range.
- Stability: Both implementations stabilize over time, with semaphores showing marginally better performance.

Comparison of the previous implementations (2)

Display Protothread Time:

- **Efficiency:** Minimal improvement in update time with semaphores.
- **Stability:** Both methods maintain initial low time, with semaphores providing slightly faster update times.

Full Circle Time:

- **Efficiency:** Significant increase in time when using semaphores.
- **Stability:** Semaphore implementation shows a higher range of times, indicating less stability.

Conclusions & Recommendations

regarding the protothread and semaphore methods

Conclusions:

Semaphores offer slight performance improvements in specific protothreads (Button, LED, Display) but at the cost of a significantly increased full circle time. General stability is maintained across most protothreads with slight improvements, except for the Full Circle Time which shows considerable instability when using semaphores.

Recommendations:

Selective Use of Semaphores: Implement semaphores selectively where minor performance improvements are crucial, such as in the LED and Display protothreads.

FreeRTOS Implementation

- In this part of the project our main purpose was to improve the task management and efficiency using FreeRTOS.

TaskHandle_t is a data type used in FreeRTOS to manage tasks

```
//Initializing Task handles  
TaskHandle_t TaskButton;  
TaskHandle_t TaskPotentiometer;  
TaskHandle_t TaskLed;  
TaskHandle_t TaskDisplay;
```

SemaphoreHandle_t is a data type used in FreeRTOS to manage semaphores

```
//Initializing Semaphore handles  
SemaphoreHandle_t sem_button;  
SemaphoreHandle_t sem_potentiometer;  
SemaphoreHandle_t sem_led;  
SemaphoreHandle_t sem_display;
```

FreeRTOS Implementation

```
void setup() {
```

```
// initializing the counting semaphores with max count 1  
sem_button = xSemaphoreCreateCounting(1, 1);  
sem_potentiometer = xSemaphoreCreateCounting(0, 1);  
sem_led = xSemaphoreCreateCounting(0, 1);  
sem_display = xSemaphoreCreateCounting(0, 1);
```

Create a counting semaphore that can be given multiple times up to a maximum count, allowing multiple tasks to access a shared resource.

```
// Creating tasks
```

```
xTaskCreate(TaskButtonCode, "Button Task", 128, NULL, 1, NULL);  
xTaskCreate(TaskPotentiometerCode, "Potentiometer Task", 128, NULL, 1, NULL);  
xTaskCreate(TaskLedCode, "LED Task", 128, NULL, 1, NULL);  
xTaskCreate(TaskDisplayCode, "Display Task", 128, NULL, 1, NULL);
```

```
xSemaphoreGive(sem_button)
```

```
}
```

Initialize a
task

Button Task Function

```
void TaskButtonCode(void *pvParameters) {  
    (void) pvParameters;  
  
    int buttonState = 0;  
    for (;;) {  
        if (xSemaphoreTake(sem_button, portMAX_DELAY) == pdTRUE) {  
            startTime = micros();  
            int currentState = digitalRead(BUTTON_PIN);  
            if (currentState != buttonState) {  
                lastDebounceTime = millis();  
            }  
            if ((millis() - lastDebounceTime) > debounceDelay) {  
                if (currentState == HIGH) {  
                    ledState = !ledState;  
                }  
            }  
            buttonState = currentState;  
            endTime = micros();  
            Serial.print("Button Task Time: ");  
            Serial.println(endTime - startTime);  
  
            xSemaphoreGive(sem_potentiometer);  
            vTaskDelay(1); // Yield to other tasks  
        }  
    }  
}
```

Potentiometer Task Function

```
// Task function for the potentiometer
void TaskPotentiometerCode(void *pvParameters) {
    (void) pvParameters;

    for (;;) {
        if (xSemaphoreTake(sem_potentiometer, portMAX_DELAY) == pdTRUE) {
            startTime = micros();
            potValue = analogRead(POTENTIOMETER_PIN);
            endTime = micros();
            Serial.print("Potentiometer Task Time: ");
            Serial.println(endTime - startTime);

            xSemaphoreGive(sem_led);
            vTaskDelay(1); // Yield to other tasks
        }
    }
}
```

LED Task Function

```
// Task function for the LED
void TaskLedCode(void *pvParameters) {
    (void) pvParameters;

    for (;;) {
        if (xSemaphoreTake(sem_led, portMAX_DELAY) == pdTRUE) {
            startTime = micros();
            analogWrite(LED_PIN, ledState ? map(potValue, 0, 1023, 0, 255) : 0);
            endTime = micros();
            Serial.print("LED Task Time: ");
            Serial.println(endTime - startTime);

            xSemaphoreGive(sem_display);
            vTaskDelay(1); // Yield to other tasks
        }
    }
}
```

OLED Task Function

```
// Task function for the display
void TaskDisplayCode(void *pvParameters) {
    (void) pvParameters;

    for (;;) {
        if (xSemaphoreTake(sem_display, portMAX_DELAY) == pdTRUE)
        {
            startTime = micros();

            if (ledState != lastLedState || potValue != lastPotValue) {
                if (ledState != lastLedState) {
                    u8x8.setCursor(0, 0);
                    u8x8.print("LED State: ");
                    u8x8.print(ledState ? "ON " : "OFF");
                    lastLedState = ledState;
                }

                if (potValue != lastPotValue) {
                    u8x8.setCursor(0, 1);
                    u8x8.print("Pot Value: ");
                    u8x8.print(potValue);
                    lastPotValue = potValue;
                }

                endTime = micros();
                Serial.print("Display Task Time: ");
                Serial.println(endTime - startTime);

                xSemaphoreGive(sem_button);
                vTaskDelay(1); // Yield to other tasks
            }
        }
    }
}
```

Challenges Encountered

Regarding the FreeRTOS Implementation

Memory Constraints:

- The Arduino board lacked sufficient memory to handle all 4 xSemaphores tasks that were necessary. Two was the maximum number of xSemaphores that the board was able to handle.

Non-functional Experiment:

- The system failed to operate correctly due to memory limitations.
- Tasks could not be executed as planned.

Future Work and Improvements

Hardware Upgrade:

- Use of a microcontroller with more memory and processing power, such as the Mega or STM32 series. These microcontrollers offer more RAM and flash memory, making them better suited for handling FreeRTOS and multiple semaphore tasks.

Enhanced Task Scheduling:

- Implement dynamic task creation and deletion to manage memory more effectively and ensure that only necessary tasks are running at any given time

Use Advanced Synchronization Mechanisms:

- Utilize message queues for efficient inter-task communication and to avoid race conditions. This method ensures that tasks can communicate safely without interfering with each other's operations

Bibliography & References

- Adam Dunkels, Protothreads: The Protothreads Library 1.4 Reference Manual, 2006
- Adam Dunkels, Oliver Schmidt, Thiemo Voigt, Muneeb Ali, Protothreads: Simplifying Event-Driven Programming of Memory-Constrained Embedded Systems, 2006
- Stoyanov, Y, RTOS: Mutex and semaphore basics, Open4Tech, (2020, March 16)
- Embedds, A comprehensive guide to RTOS in embedded systems: Types, benefits, choosing the right RTOS, and more, Embedds, Retrieved June 9, 2024



**Thank you
for you attention!**

Any Questions ?